HO CHI MINH CITY NATIONAL UNIVERSITY
UNIVERSITY OF NATURAL SCIENCES
FACULTY OF INFORMATION TECHNOLOGY

FINAL PROJECT REPORT
SUBJECT: INTRODUCTION TO MACHINE LEARNING

INSTRUCTOR:
Bùi Tiến Lên

22KHDL2 - Group:
22127069 - Nguyễn Đặng Hoàng Dinh
22127107 - Nguyễn Thế Hiển
22127139 - Hoàng Duy Hưng
22127198 - Phạm Anh Khoa

Ho Chi Minh City, 4/2025

TABLE OF CONTENTS

WORK ASSIGNMENT TABLE		**Level of completion**
22127069 - Nguyễn Đặng Hoàng Dinh	Model Design and Implementation Model Performance Evaluation Report Writing Video Making	100%
22127139 - Hoàng Duy Hưng	Data collection, processing Report Writing Front-end for Home page and Genre	100%
22127198 - Phạm Anh Khoa	Code the frontend of the Chatbot Design Slides Report Writing	100%
22127107 - Nguyễn Thế Hiển	Code frontend for Movie Description Design Slide Report Writing Improve + testing	100%

I. Topic ideas and objectives

1. Idea:

- **Real-world problem:** Users have difficulty finding the right movie due to
  - Users often struggle to find movies due to the diversity of platforms and the overwhelming number of options.
  - Difficulty in expressing search queries.
  - Lack of an intelligent recommendation system that combines multiple criteria.
- **Proposed solution:**
  - Building a smart movie recommendation system that combines.
  - Utilize pre-trained AI to comprehend natural language requests (multilingual Vietnamese/English).
  - Rule-based filtering from the local dataset.
  - TMDB API to update new data and add information (trailers, posters…).
  - Fallback AI (ChatGPT) to handle complex cases.
  - Fuzzy matching to handle input bias.

2. Target:

- **Technical:**
  - Successfully integrated 3 data sources: Local dataset, TMDB API, OpenAI.
  - Using Prompt Engineering to Understand Natural Language Requests and AI Fallbacks.
  - Handle multilingual input (Vietnamese/English).
  - Achieved >80% accuracy on test cases.
- **Application:**
  - Chatbot web interface helps users look up.
  - Response time < 10s for each request.
  - Support searching by multiple criteria (actors, genres, directors,...).

II. Analyze and define the problem

1. Key challenges:

- **Natural language processing:**
  - Diverse inputs: "best Tom Hanks movies", "best action movies 2023", misspellings, multiple languages.
  - Need to accurately identify entities (person name, genre, actor).
- **Multi-source data integration:**
  - Local dataset (movie.csv) may be missing new movies
  - TMDB API has a different rating than the local dataset.
- **System performance:**
  - Need to balance accuracy and response time.

    - Handle multiple requests simultaneously.

2. Dataset analysis

- **movie.csv:**
    - 1000+ movies with attributes: Title, Genre, Actors, Director, Ratings.
    - Lack of new movies after 2020.
    - Inconsistent genre formatting (e.g., "sci-fi" vs. "science fiction").
- TMDB:
    - 500,000+ movies, but need API key
    - Real-time data, but response speed depends on the network.

III. Proposed solution model

1. Data collection and processing

a. Data collection

Concept

In this project, the data acquisition step is the foundation for building a movie information management and display system. We use two main methods:

- Crawling with Scrapy
- Query TMDB API

Data Source

- **TMDB (The Movie Database) official website:** provides a complete API of movie-related information, including title, description, director, actors, genre, reviews, revenue, and many other metadata fields**.**
- **Other sites:** Including IMDB and Rotten Tomatoes. Sometimes they add additional information (like posters or trailers), but in this project, all data is fetched via the TMDB API.

Scrapy Tool

- Scrapy is a dedicated Python framework for web crawling & scraping, allowing to define “spiders” to automatically collect data from HTML pages.

- Project configuration:
    - settings.py: Set up user-agent, robots.txt compliance, and processing pipelines.
    - spiders/movie_spider.py: main spider definition, starting URL details, how to parse pages, and extract raw links to movie details.

- Pipeline: Get a list of movie detail URLs, then push it to the API call stage to get the structured data.

TMDB API

- API Key: The identifier (TMDB API key) is provided after creating a developer account on TMDB.
- Endpoints used:
  - /search/movie: Search movies by name or keyword.
  - /movie/{movie_id}: Get detailed information about each movie (including title, overview, genres, runtime, release_date, vote_average, etc.).
  - /movie/{movie_id}/credits: Get cast & crew info.
- General parameters:
  - api_key: Required for all requests.
  - language=vi-VN: Priority is given to returning Vietnamese results (if available).
  - append_to_response=videos, images: If needed, get trailer and poster at the same time.

Data collection process

Initialize Scrapy Spider: The Spider launches on the movie category page, collects detailed links of each movie.

- Call TMDB API
  - For each movie_id obtained, request the endpoints /movie/{movie_id} and /credits.
  - Temporary storage
  - Raw data is saved to a .csv file for testing and debugging.

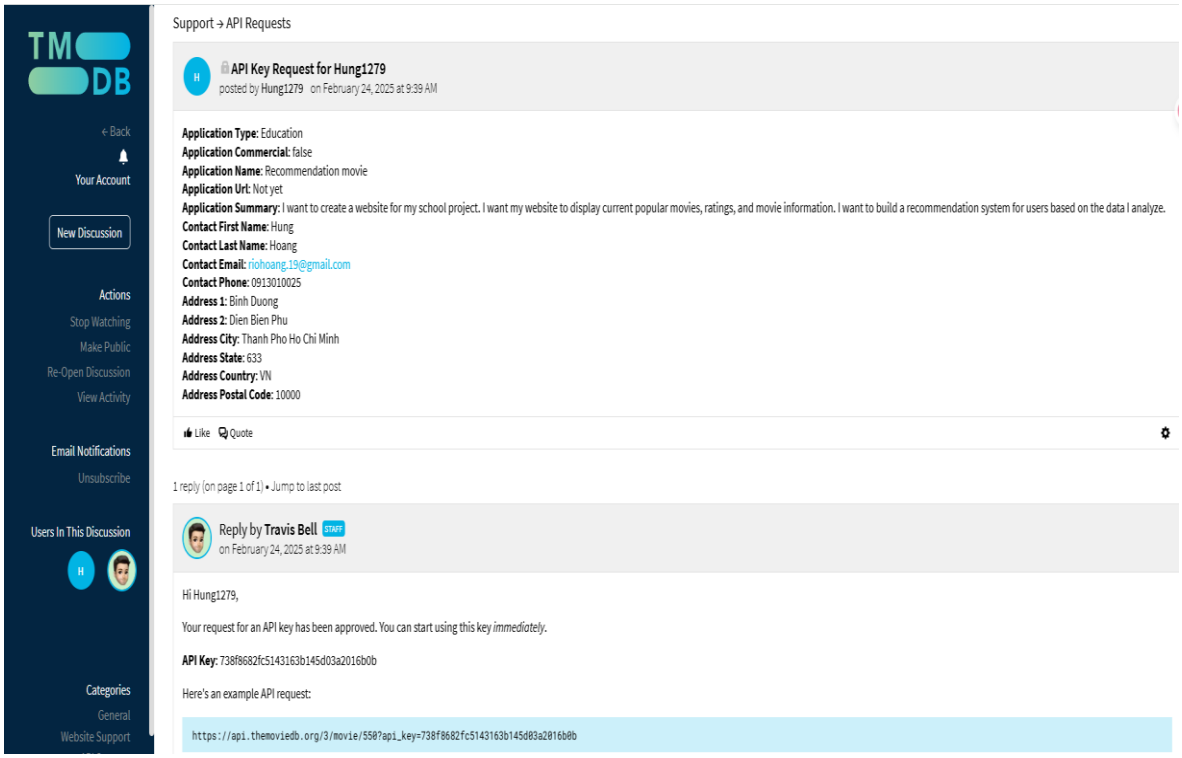
- Post processing

  - Normalize field names, check for null values, and remove duplicates.

b. Reality

1. TMDB

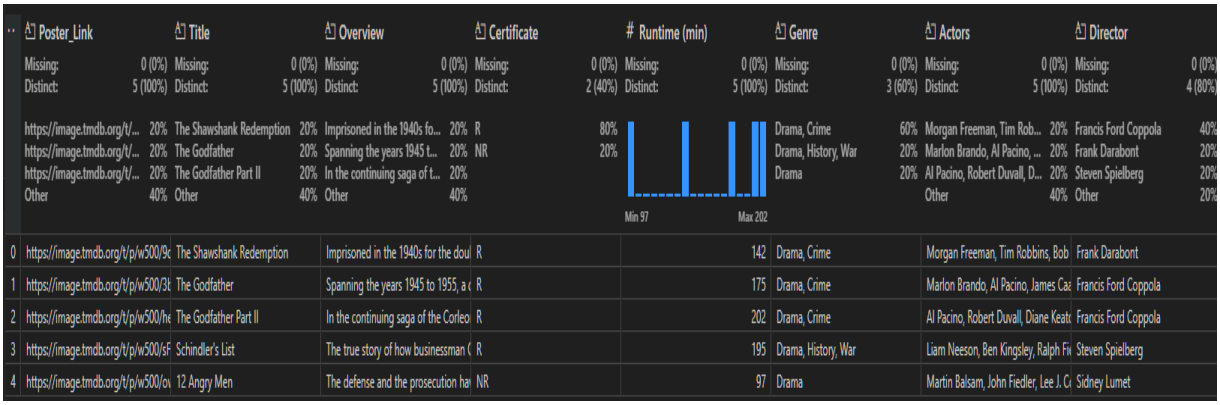
- First, ask for the TMDB API permission



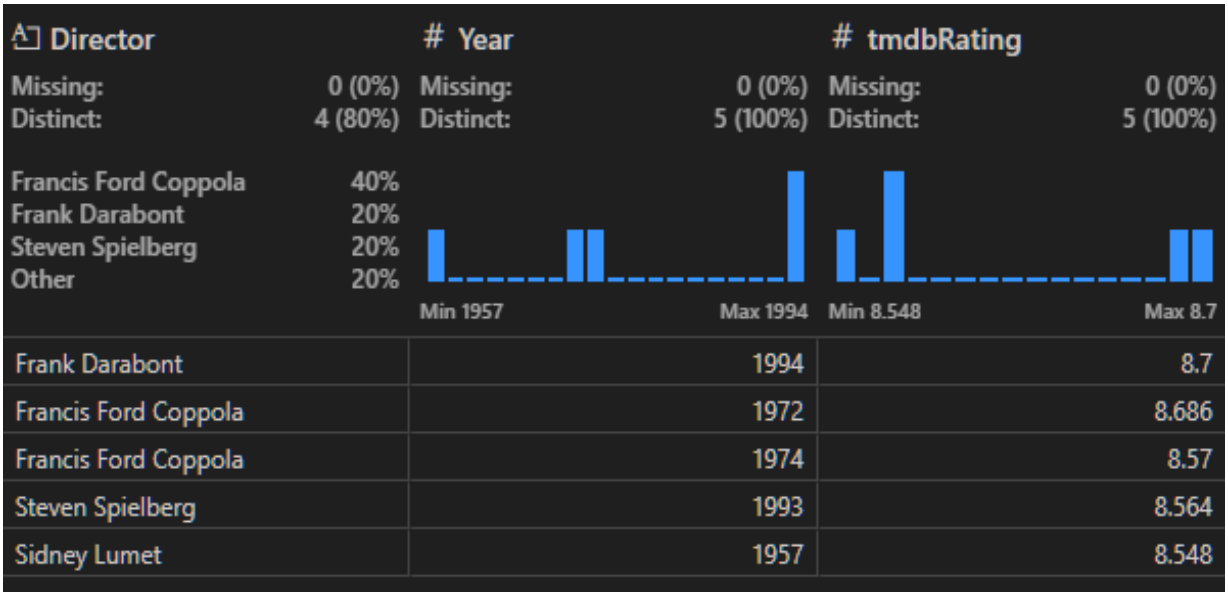
- Understanding the importance of API keys, the team asked the website for permission for learning purposes, and was approved by the website.
- After getting the API key, we will work on Python to get the necessary information columns.

```python
movie_data = {
    'Poster_Link': poster_url,
    'Title': details.get('title', ''),
    'Overview': details.get('overview', ''),
    'Certificate': cert,
    'Runtime (min)': details.get('runtime', 0),
    'Genre': ', '.join(genres),
    'Actors': ', '.join(actors),
    'Director': ', '.join(directors),
    'Year': details.get('release_date', '')[:4],
    'tmdbRating': details.get('vote_average', 0)
}
```

- Important properties to fetch include Poster_link, Title, Overview, Certificate, Runtime, Genre, Actors, Director, Year, and tmdbRating.
- After collecting data, save it to the TMDB.csv file for later processing.



	Poster_Link	Title	Overview	Certificate	Runtime (min)	Genre	Actors	Director
	Missing: 0 (0%) Distinct: 5 (100%)	Missing: 0 (0%) Distinct: 5 (100%)	Missing: 0 (0%) Distinct: 5 (100%)	Missing: 0 (0%) Distinct: 2 (40%)	Missing: 0 (0%) Distinct: 5 (100%)	Missing: 0 (0%) Distinct: 3 (60%)	Missing: 0 (0%) Distinct: 5 (100%)	Missing: 0 (0%) Distinct: 4 (80%)
	https://image.tmdb.org/t/... 20%	The Shawshank Redemption 20%	Imprisoned in the 1940s fo... 20%	R 80%	Min 97 Max 202	Drama, Crime 60%	Morgan Freeman, Tim Rob... 20%	Francis Ford Coppola 40%
	https://image.tmdb.org/t/... 20%	The Godfather 20%	Spanning the years 1945 t... 20%	NR 20%		Drama, History, War 20%	Marlon Brando, Al Pacino, ... 20%	Frank Darabont 20%
	https://image.tmdb.org/t/... 20%	The Godfather Part II 20%	In the continuing saga of t... 20%			Drama 20%	Al Pacino, Robert Duvall, D... 20%	Steven Spielberg 20%
	Other 40%	Other 40%	Other 40%				Other 40%	Other 20%
0	https://image.tmdb.org/t/p/w500/9c	The Shawshank Redemption	Imprisoned in the 1940s for the dou	R	142	Drama, Crime	Morgan Freeman, Tim Robbins, Bob	Frank Darabont
1	https://image.tmdb.org/t/p/w500/3t	The Godfather	Spanning the years 1945 to 1955, a	R	175	Drama, Crime	Marlon Brando, Al Pacino, James Ca	Francis Ford Coppola
2	https://image.tmdb.org/t/p/w500/he	The Godfather Part II	In the continuing saga of the Corleo	R	202	Drama, Crime	Al Pacino, Robert Duvall, Diane Keat	Francis Ford Coppola
3	https://image.tmdb.org/t/p/w500/sF	Schindler's List	The true story of how businessman (	R	195	Drama, History, War	Liam Neeson, Ben Kingsley, Ralph Fi	Steven Spielberg
4	https://image.tmdb.org/t/p/w500/ox	12 Angry Men	The defense and the prosecution ha	NR	97	Drama	Martin Balsam, John Fiedler, Lee J. C	Sidney Lumet



Director		Year		tmdbRating	
Missing:	0 (0%)	Missing:	0 (0%)	Missing:	0 (0%)
Distinct:	4 (80%)	Distinct:	5 (100%)	Distinct:	5 (100%)
Francis Ford Coppola	40%	Min 1957	Max 1994	Min 8.548	Max 8.7
Frank Darabont	20%				
Steven Spielberg	20%				
Other	20%				

Frank Darabont	1994	8.7
Francis Ford Coppola	1972	8.686
Francis Ford Coppola	1974	8.57
Steven Spielberg	1993	8.564
Sidney Lumet	1957	8.548

- Since the basic TDMB page already has the most important properties, from the following sections, only take points from other websites.

2. IMDB

- To work with IMDB, the team decided to use Scrapy. However, IMDB has measures in place to block its data collection. We had to make a few changes before scraping could be run.

```
# Obey robots.txt rules
ROBOTSTXT_OBEY = False
```

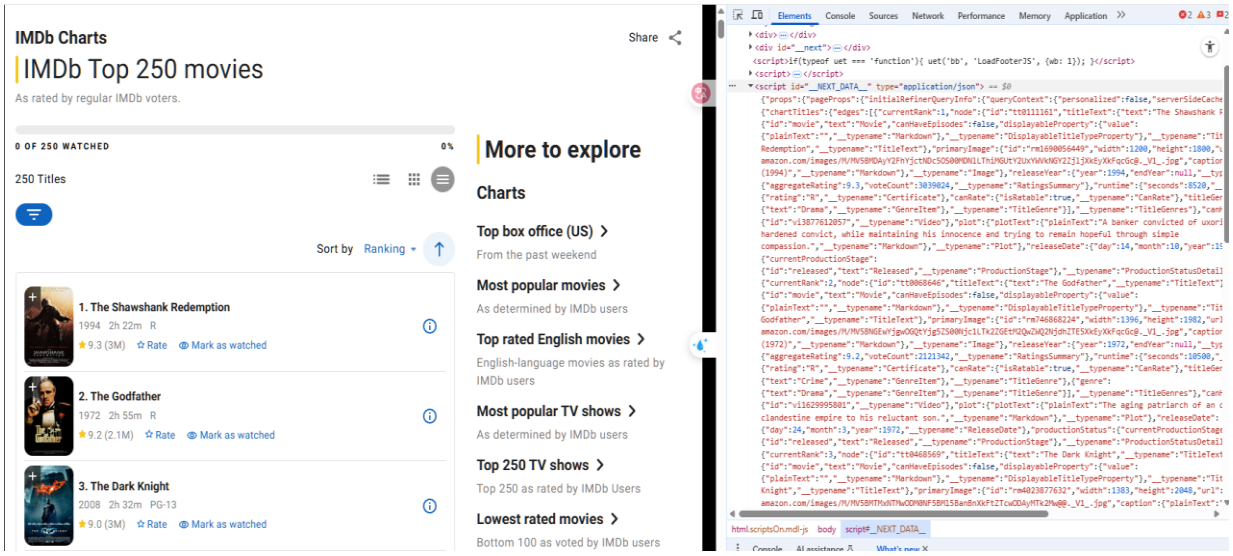
- This is a setting that determines whether Scrapy will respect your website's robots.txt file or not.
- robots.txt is a file that each website can use to specify which parts of the site crawlers/bots are allowed or not allowed to access.

When placing:

- ROBOTSTXT_OBEY = True: Scrapy will respect the rules in robots.txt, and ignore URLs that are not allowed to be crawled.
- ROBOTSTXT_OBEY = False: Scrapy will ignore robots.txt, and crawl all pages it can access, including those banned in robots.txt.

```
DOWNLOADER_MIDDLEWARES = {
    'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware': None,
    'scrapy_user_agents.middlewares.RandomUserAgentMiddleware': 400,
}
```

- This is the **middleware configuration section that handles when Scrapy sends and receives requests/responses.**
- Use **Random User-Agent** to impersonate a different browser each time a request is sent.
  => This helps **avoid being detected as a bot**, **reduces the chance of being blocked**, and **simulates real user access behavior**.
- After passing the blocking step, our goal will be to find the top 250 movies with the highest IMDb scores in history and save them to a CSV file.



- After accessing the IMDB top 250 movie website, we realize that all the data is stored in an HTML tag. We will filter out each tag of that data to get the movie name and score of that movie.

```python
def parse(self, response):

    raw_data = response.css("script[id='__NEXT_DATA__']::text").get()

    json_data = json.loads(raw_data)

    needed_data = json_data['props']['pageProps']['pageData']['chartTitles']['edges']

    info = Info()

    for movie in needed_data:
        info['Title'] = movie['node']['titleText']['text'],
        info['imdbRating']  = movie['node']['ratingsSummary']['aggregateRating'],

        yield info
```

- We will get 2 columns of data: Title and IMDb Rating.

	Title	imdbRating
	Missing: 0 (0%) Distinct: 250 (100%)	Missing: 0 (0%) Distinct: 13 (5%)
	250 Distinct values	Min 8.0 Max 9.3
0	The Shawshank Redemption	9.3
1	The Godfather	9.2
2	The Dark Knight	9.0
3	The Godfather Part II	9.0
4	12 Angry Men	9.0
5	The Lord of the Rings: The Return of	9.0
6	Schindler's List	9.0
7	Pulp Fiction	8.9
8	The Lord of the Rings: The Fellowshi	8.9
9	The Good, the Bad and the Ugly	8.8
10	Forrest Gump	8.8
11	The Lord of the Rings: The Two Towe	8.8
12	Fight Club	8.8
13	Inception	8.8
14	Star Wars: Episode V - The Empire St	8.7
15	The Matrix	8.7
16	Goodfellas	8.7

Rotten tomatoes

- We will use the same approach as we used with IMDB. Our target will be the top 300 movies with the highest Rotten Tomatoes rating.

300 BEST MOVIES OF ALL TIME

Welcome to the 300 highest-rated best movies of all time, as reviewed and selected by Tomatometer-approved critics and Rotten Tomatoes users.

How were the movies selected and ranked? First, every movie here is Certified Fresh. (Learn more on the About page.) Then we applied our recommendation formula, which considers a movie's Tomatometer rating with assistance from the Popcornmeter, illuminating beloved sentiment from both sides. Critics-certified, audience-approved!

Other factors weighing into the recommendation formula: a movie's number of critics reviews, the number of Audience Score votes, and its year of release. An editorial pass is reserved to finesse the final list, which included minimum thresholds for each of these data points.

1.	**97%** The Godfather (1972)
2.	**99%** Casablanca (1942)
3.	**99%** L.A. Confidential (1997)
4.	**100%** Seven Samurai (1954)
5.	**99%** Parasite (2019)
6.	**98%** Schindler's List (1993)
7.	**96%** Top Gun: Maverick (2022)
8.	**98%** Chinatown (1974)
9.	**99%** On the Waterfront (1954)
10.	**100%** Toy Story 2 (1999)

- However, a little different from IMDB is that all data is stored in one HTML tag, but on this page, we have to find each tag that stores the appropriate data and then retrieve it.

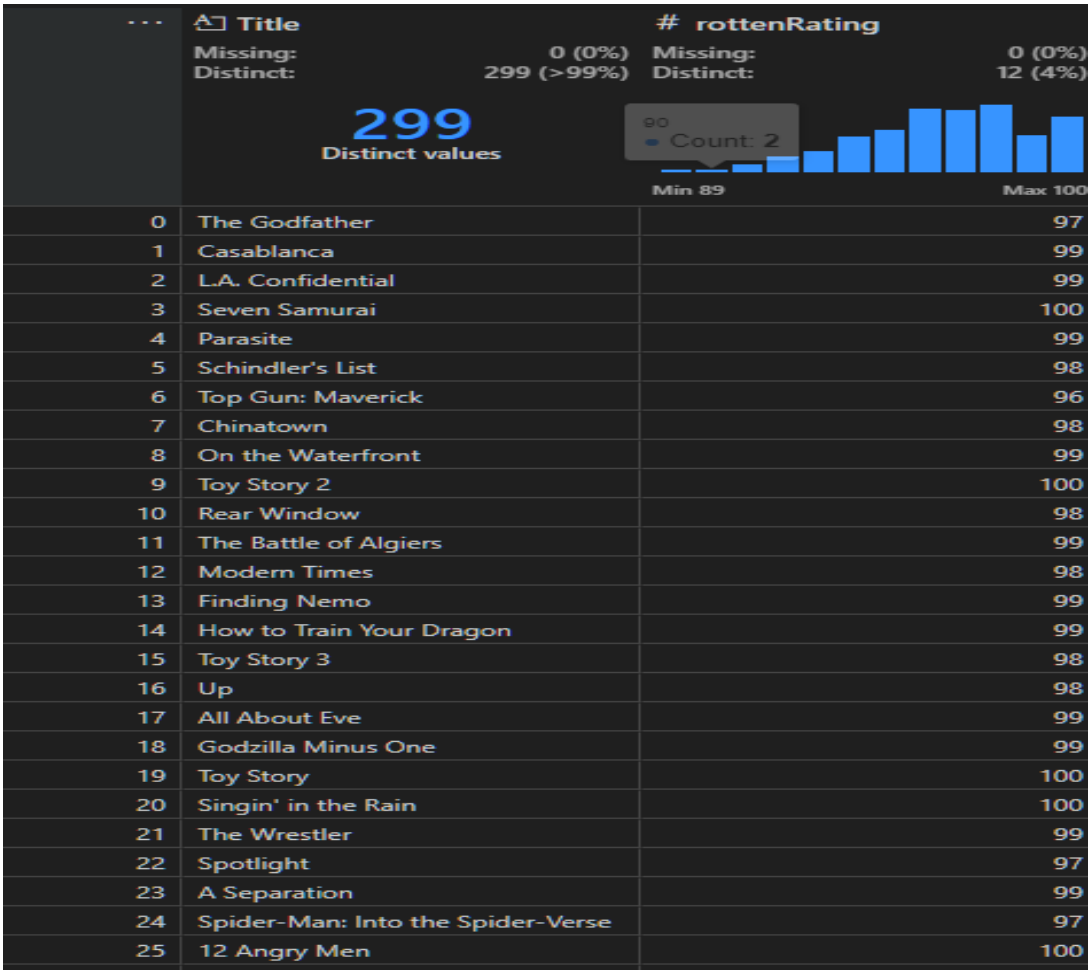
```
class RottenSpider(scrapy.Spider):
    name = "rotten"
    allowed_domains = ["editorial.rottentomatoes.com"]
    start_urls = [
        "https://editorial.rottentomatoes.com/guide/best-movies-of-all-time/",
    ]

    def parse(self, response):
        for movie in response.css("p.apple-news-link-wrap.movie"):
            # Title
            raw_title = movie.css("span.details a.title::text").get()
            if not raw_title:
                continue
            title = raw_title.strip()

            # Score
            raw_score = movie.css("span.score strong::text").get()
            score = raw_score.replace("%", "").strip() if raw_score else None

            item = RottenItem()
            item["Title"] = title
            item["rottenRating"] = score
            yield item
```

- After getting it, save it to a CSV file for later processing..



	Title	rottenRating
	Missing: 0 (0%) Distinct: 299 (>99%)	Missing: 0 (0%) Distinct: 12 (4%)
	299 Distinct values	Count: 2 Min 89 Max 100
0	The Godfather	97
1	Casablanca	99
2	L.A. Confidential	99
3	Seven Samurai	100
4	Parasite	99
5	Schindler's List	98
6	Top Gun: Maverick	96
7	Chinatown	98
8	On the Waterfront	99
9	Toy Story 2	100
10	Rear Window	98
11	The Battle of Algiers	99
12	Modern Times	98
13	Finding Nemo	99
14	How to Train Your Dragon	99
15	Toy Story 3	98
16	Up	98
17	All About Eve	99
18	Godzilla Minus One	99
19	Toy Story	100
20	Singin' in the Rain	100
21	The Wrestler	99
22	Spotlight	97
23	A Separation	99
24	Spider-Man: Into the Spider-Verse	97
25	12 Angry Men	100

a. Data processing

- Since the data is collected from 3 separate sites. We have to merge it into 1 big file and handle any problems that may arise while merging the files. And the direction of the group will be to merge the files based on the same movie name.
- **Overview of the data** we have: 3 files, including
    - TMDB.csv: Include Poster_Link, Title, Overview, Certificate, Runtime (min), Genre, Actors, Director, Year,tmdbRating.
    - imdb.csv: Include Title, imdbRating.
    - rotten.csv: Include Title, rottenRating.
- What we are going to do now is
    - Merge 3 files.
        - Treatment method
            - Use **fuzzy matching** (fuzzywuzzy library) to match non-exactly identical titles between datasets
            - Based on **token_set_ratio** with a **90%** accuracy threshold for title matching.
            - Data from IMDb and Rotten Tomatoes are **appended to the original TMDB** set by fuzzy matching results.
        - Read 3 CSV data files.

        - Apply fuzzy matching to find similar titles between TMDB, IMDb, and Rotten Tomatoes.
        - Use pd. merge to append data columns from IMDb and Rotten to the main table.
        - Remove unnecessary intermediate columns.

Export the final result to a merged_result.csv file. The new file will contain the columns of the TMDB.csv file and add 2 columns, imdbRating and rottenRating, based on the 2 corresponding files, imdb.csv and rotten.csv.

- Data Overview.
    - **Remove duplicate rows**
        - Use data.duplicated().sum() to check the number of duplicate rows.
        - If so, use data.drop_duplicates() to remove them.
    - **Check the data type of each column**
        - Print out the data. dtypes to see what type each column is (number, string, etc).
    - **Calculate the missing data rate**
        - Use data.isnull().mean() * 100 to calculate the percentage of missing values for each column.
    - **Remove columns with too high a percentage of missing data**
        - Write a function drop_missing_features() to drop columns with more than 75% missing values (threshold=75.0).
    - **Descriptive statistics for numeric columns (float/int):**
        - Filter columns of type float64, int64 and calculate statistics::
            - Missing_ratio
            - Min, Max
            - Quartile values: Q1 (25%), Median (50%), Q3 (75%)

The result is a summary table (num_col_info_df) for the quantitative variables.

    - **Analysis of categorical variables:**
        - Filter non-numeric columns (exclude=['float64', 'int64'])
        - For each column
        - Calculate the shortage rate
        - Count the number of unique values (class labels)

The result is a summary table (cat_col_info_df) for the nominal variables.

    - **Data cleaning**

The goal of this section is to handle missing movie data by automatically filling in the missing fields (poster, description, actors, director, duration, certification, genre, rating, etc.) via the API of the TMDB (The Movie Database) website.

Steps to follow

- **Handling missing values:**
  Fill in the missing numeric fields like **imdbRating** and **rottenRating** with 0. This happens when merging 3 files using TMDB as the original file. The number of rows in the TMDB file is 500. While IMDB file is 250 and Rotten is 300. Merging files will cause some rows to be missing data. And filling in the missing scores with 0 will help group them, and 0 represents no data.
- **Search and retrieve movie data**:
  - Use the TMDB API to find movies by title..
  - Get movie details (poster, description, duration, rating, genre...).
  - Get the cast list, director, and rating certificate.
- **Fill in the missing information for each row:** This is to add cells other than the score column; any missing columns will be filled in**.**
  - If a field is empty, the system will automatically call the API and fill in the data.
  - There is a 0.25s delay per call to avoid exceeding API limits.

  Write the results to a movie.csv file with all the data fields.

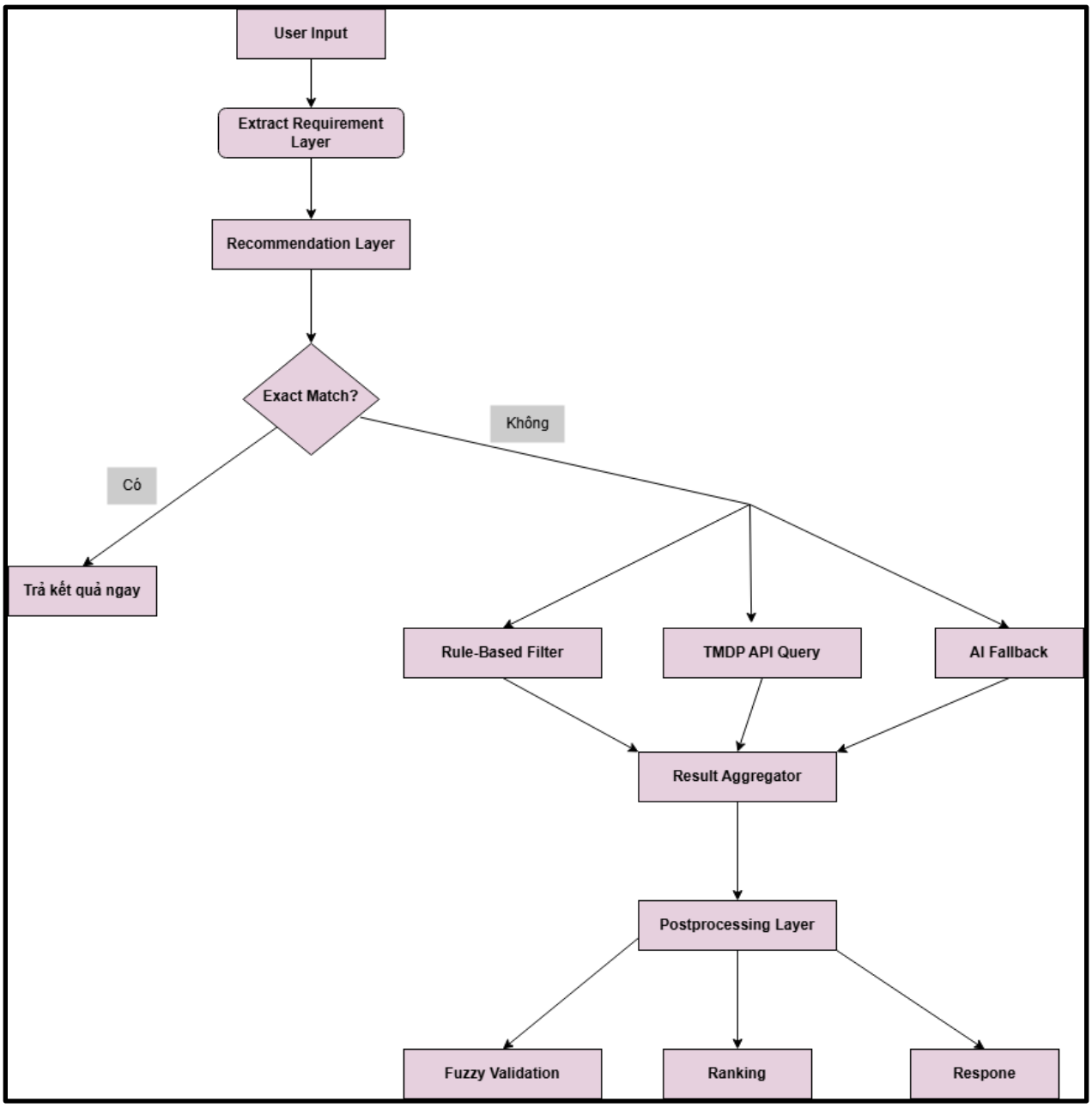
- **Results achieved**
  - Complete movie data set with complete and accurate information.
  - Consistent data format, ready for analysis, visualization, or integration into a web interface.
- **Difficulties and solutions**
  - Some movie titles not found in TMDB: handled by keeping the original data stream and error message.
  - API has call limits: add a timeout between calls to ensure no blocking.

IV. Model design and implementation

4.1.Overall architecture:

The system is designed according to a **hybrid** model combining 3 main processing layers:

- **Extract requirement Layer**
- **Recommendation Layer**
- **Postprocessing Layer**



4.2. Details of each component

4.2.1. Request Extraction Class

Goal: Convert natural user input into JSON structure, extract necessary entities: **actors, genres**,... so that the program can process automatically and accurately.

Detailed Workflow:

Step 1: Call OpenAI API

- Prompt Engineering:
  Tweak Prompt to extract user requests in JSON structure with necessary entities: actors, genres,....
- Model: gpt-4o-mini with temperature = 0.3 to balance creativity and precision.
- Response Format: Ép kiểu JSON nghiêm ngặt

Step 2: Standardize the genre

- Mapping Dictionary: 35+ Vietnamese-English genre pairs

```
genre_mapping = {
    "kinh dị": "horror",
    "hài": "comedy",
    # ...
}
```

- Accurate mapping:
  We will map the user's Vietnamese category requests to English using the created Mapping Dictionary accurately.

```
# Tìm ánh xạ chính xác
if normalized_genre in genre_mapping:
    mapped_genres.append(genre_mapping[normalized_genre])
```

- Fuzzy matching for unspecified genre:

```
# Fallback: Tìm ánh xạ gần đúng nhất
closest_match = difflib.get_close_matches(
    normalized_genre,
    genre_mapping.keys(),
    n=1,
    cutoff=0.7
)
```

4.2.2. Recommendation Layer

First we will handle the case where the user enters a specific movie name:

- **Goal:** Detect when the user types in the correct (or close to) movie name, to return the correct result without going through the entire pipeline.
- **How it works:**
  - Normalize input (trim + lowercase)..
  - Use difflib.get_close_matches (cutoff 0.8) to find the closest movie name in the dataset.

- If found, calculate the similarity ratio (SequenceMatcher.ratio()) and only accept when ≥0.8 → return the original movie name.
- If not in the dataset, call the TMDB API (get_movie_id_tmdb) to find the ID and fetch the title from TMDB.

- **Impact on performance:**
  - Reduced cost and latency (no GPT calls or complex filtering) when the user already knows the specific movie name.

The strategy proposes 3 levels:

a. Rule-Based Filtering

- Mechanism:
  - Filter movies based on actors, genres, directors from local dataset.
  - Based on the results of the request **extraction layer**, we use **fuzzy matching** to match the results with data in the **local dataset** (find the name closest to the user's input).
  - After successful matching, create a **logical mask** to filter the movies in the local dataset containing the mapped entities.

```
for genre in genres:
    # Tìm genre gần nhất trong danh sách đã tách
    closest_genre = difflib.get_close_matches(
        genre.lower(),
        list(all_genres),
        n=1,
        cutoff=0.6  # Giảm cutoff để bắt "scifi" -> "sci-fi"
    )

    if closest_genre:
        print(f"Mapped genre: {genre} -> {closest_genre[0]}")
        genre_mask |= df["Genre"].str.lower().str.contains(closest_genre[0], na=False)

mask &= genre_mask
```

- Optimization:
  - Use **LRU Cache (size 500)** to speed up repeated queries.

b. TMDB API Intergration

- Mechanism:
  - Get movies by actor from TMDB, limited to 10 movies/actor.

```python
@lru_cache(maxsize=200)
def get_movies_by_actor(actor_name):
    """Lấy phim từ TMDB theo diễn viên"""
    url = f"https://api.themoviedb.org/3/search/person?api_key={TMDB_API_KEY}&query={actor_name}"
    response = requests.get(url).json()

    if response["results"]:
        actor_id = response["results"][0]["id"]
        movies_url = f"https://api.themoviedb.org/3/person/{actor_id}/movie_credits?api_key={TMDB_API_KEY}"
        movies_response = requests.get(movies_url).json()
        return tuple(movie["title"] for movie in movies_response.get("cast", []))
```

- Asynchronous processing:
  - Use caching (maxsize=200) to reduce latency.

c. AI Fallback

- Activated when:
  - **Rule-based** and **TMDB** do not return results.
- Prompt Engineering:

  Use Prompt technique to get movies from **local dataset** for complex requests.

```python
prompt = f"""
Danh sách phim: {', '.join(df['Title'].tolist()[:200])}
Thể loại: {', '.join(df['Genre'].unique()[:50])}
Yêu cầu: {user_input}
Đề xuất phim kết hợp các yếu tố trên (nếu có). Chỉ trả về tên phim.
"""
```

- Processing output:
  - Standardize movie names, remove special characters.
  - **Fuzzy Matching** to map with dataset.

3.2.2. Postprocessing Layer

1. **Aggregation:**
   - **Combine results from 3 sources: Local, TMDB, AI.**

```python
combined = list(set(local_results + tmdb_results + ai_results))
```

2. **Fuzzy Validation:**
   - **Make sure all movies exist in the dataset**

```python
valid_movies = [title for title in combined if title in df["Title"]]
```

3. **Ranking:**
   - Sort movies by IMDB Rating (high -> low)

4. **Result limits:**
    - Returns top 5 highest rated movies

V. Model performance evaluation

5.1. Evaluation method:

a. Precision và Recall:
- **Precision (Accuracy):** Measures the percentage of movies that actually match among the suggested movies.
  Reason: Users are usually only interested in the first few results, need to optimize the quality of the top 5.
- **Recall (Coverage):** Measures the system's ability to find all relevant movies present in the expected results.
  Reason: Ensure the system does not miss high quality movies.
- **Precision-Recall Combination**: Balance between precision and coverage, avoiding one-way optimization.

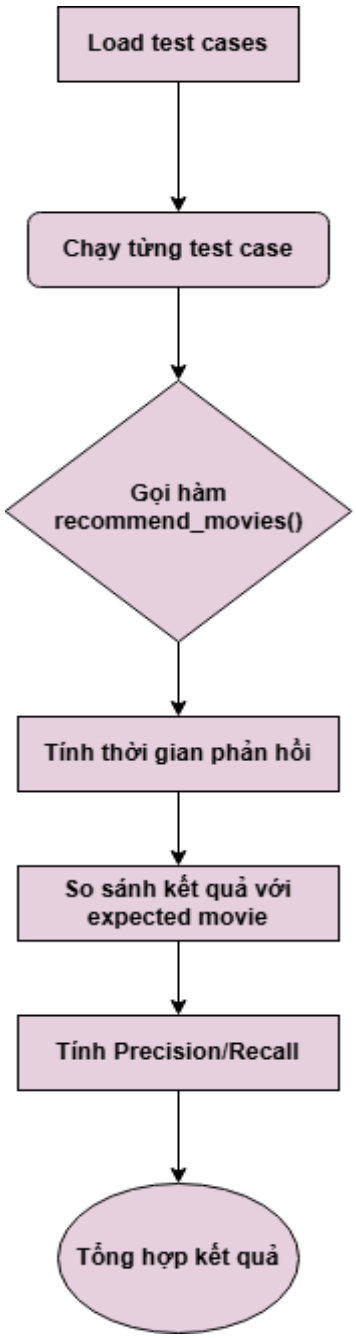
b. Response time:
- **Meaning:** Time from when the user enters a request until the result is received.
- **Reason for measurement:** Ensure smooth user experience, High response time (>10s) can make users impatient.
  Detect processing bottlenecks (API calls, natural language processing).

c. Diverse test case set:
- Includes 30 test cases with scenarios.
  Input:
  - Complex synonyms/genres
  - Multilingual input (Vietnamese/English)
  - Combine mixtures of entities actors, genres, directors,…

  Output (Labeled data):
  The list of movie names in the local dataset matches the user's request.
  Reason: Simulate real user behavior.

5.2. How the rating model works:

1. Processing flow:



2. Calculation formula:

- **Precision:**
  - **Formula:**

$$\text{Precision} = \frac{\text{Số phim đề xuất đúng}}{\text{Tổng số phim được đề xuất}}$$

Meaning: Precision reflects the proportion of movies that actually match the recommended list. For example, if the system recommends 5 movies and 3 of them match the expected list, Precision = 3/5 = 0.6.

Why it's used: This metric helps evaluate the system's ability to avoid false recommendations, ensuring users receive high-quality results.

- **Recall (Coverage):**
  - **Formula:**

$$\text{Recall} = \frac{\text{Số phim đề xuất đúng}}{\text{Tổng số phim kỳ vọng}}$$

Meaning: Recall measures the ability to find all the matching movies in the expected dataset. For example, if there are 5 expected movies and the system finds 3, Recall = 3/5 = 0.6.
Why use: This metric measures how comprehensive the system is in exploring suitable options.

3. Analyze how it works:

The evaluation process is carried out through the following steps:

a. Prepare test cases:

- Test cases are stored in JSON files, each test case includes:
  - input: User request (e.g. "Tom Hanks action movies").
  - expected_movies: List of expected movies.

b. Run the assessment:

- For each test case, the system calls the recommend_movies() function to get the list of recommendations.
- Precision and Recall are calculated by comparing the proposed list with the expected list.
- Response time is measured from calling the function to receiving the result.

c. Error handling:

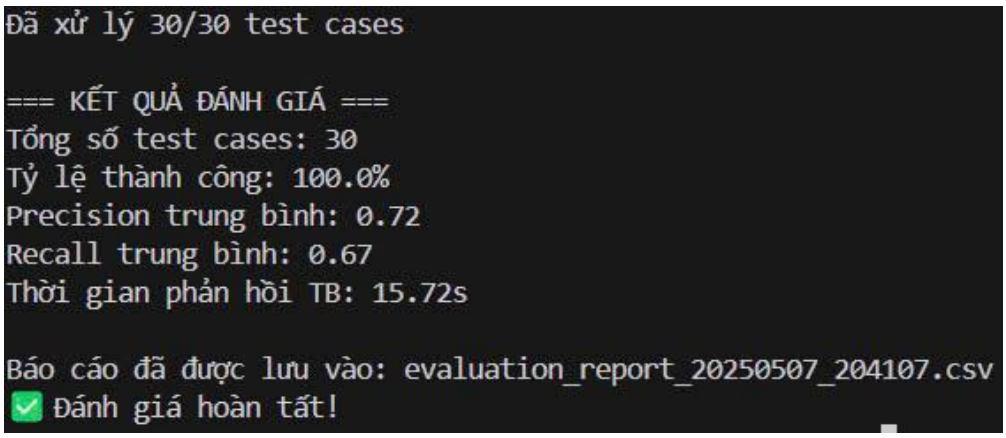
- If there is a runtime error (e.g., API connection failed), the system acknowledges it and continues processing other test cases.

d. Summary of results:

- Results are saved to CSV file, including: Precision, Recall, response time, recommendation list and expectations.
- A summary report is printed out, including average metrics and success rates.

4. Detailed analysis of the results obtained:

Here is the resulting image of the average scores of Precision, Recall and response time of 30 test cases on the terminal:



The above results show that:

- Average precision: 0.72
  - **Comment:** The system correctly recommends about 72% of the movies in the returned list.
  - **Cause:**
    - Some movies are recommended based on AI fallback and tmdb does not control imdb rating priority which leads to errors.
    - The mapping of categories/attributes from Vietnamese to English is not complete and optimal.
- Average recall: 0.67
  - **Comment:** The system only found 67% of the movies in the expected list.
  - **Cause:**
    - Limited dataset (only 250 movies), resulting in lack of popular movies.
    - Some requests are quite complex and therefore depend on TMDB API and AI Fallback: If the API does not return enough information, the system cannot make complete and accurate recommendations.

- Average response time: 15.72 seconds
  - **Comments:** Response time is quite high, affecting user experience.
  - **Cause:**
    - Some movies have quite complex requirements, leading to calls to many external APIs (TMDB), OpenAPI, increasing latency.
      - Success rate: 100%
        - **Comment:** All test cases are processed without runtime errors.
        - **Limitation:** This ratio does not reflect the quality of the proposal, only shows the system's technical stability.

5. Suggested improvements

- Increased accuracy:
  - Improve category/attribute mapping using multilingual dictionaries.
  - Train NLP models (like BERT) to understand user request semantics.
- Reduce response time:
  - Implement caching for TMDB API queries.
  - Optimize fuzzy matching algorithm using RapidFuzz library instead of difflib.
- Expand dataset:
  - Integrate additional data sources (e.g. IMDb, Netflix) to increase coverage.
- Improve reviews:
  - Add more diverse test cases.
  - Use F1-Score (harmonic average of Precision and Recall) for overall evaluation.

VI. Real-life applications and demos

1. The level of model integration into the web application

a. Application architecture

The system is built according to the **client-server** model:

- **Frontend:** Use HTML/CSS/JavaScript to handle the user interface, display results and interact with the backend via AJAX.
- **Backend:** Implemented using FlaskAPI, which provides two main endpoints:
  - **/recommend (POST):** Receive requests from users and return a list of recommended movies.
  - **/movie/<movie_title> (GET):** Get detailed information about a specific movie (poster, rating, trailer, actors).

b. Integrated flow

- Step 1: The user enters a request into the chatbot on the interface. The frontend sends a POST request to the /recommend endpoint with JSON content:

```
{ "query": "phim hành động của Tom Hanks" }
```

- Step 2: Backend processes the request using the recommend_movies() function, combining rule-based filtering, TMDB and AI (GPT-4). The result is returned

in JSON format:

```
{
  "recommended_movies": [
    { "title": "Forrest Gump", "poster": "...", ... },
    ...
  ]
}
```

- The frontend displays the list of movies as movie image cards inside the chatbot frame. When the user clicks on a movie, the frontend calls the /movie/<movie_title> endpoint to get the details.

c. Error handling and security

- CORS: Use flask-cors to allow access from domain http://127.0.0.1:5502, avoid cross-origin blocking error.
- Error handling:
  - If the user does not enter a request, the backend returns a 400 error code with the message "Query is required".
  - If the movie does not exist, return a 404 code and a message saying "Movie not found".
  - Server errors (e.g. TMDB API not responding) are caught with try-except and return code 500.

d. Integrate search history

- Storage: History is stored client-side using localStorage to ensure privacy.
- Display: Users can review recent queries in a sidebar

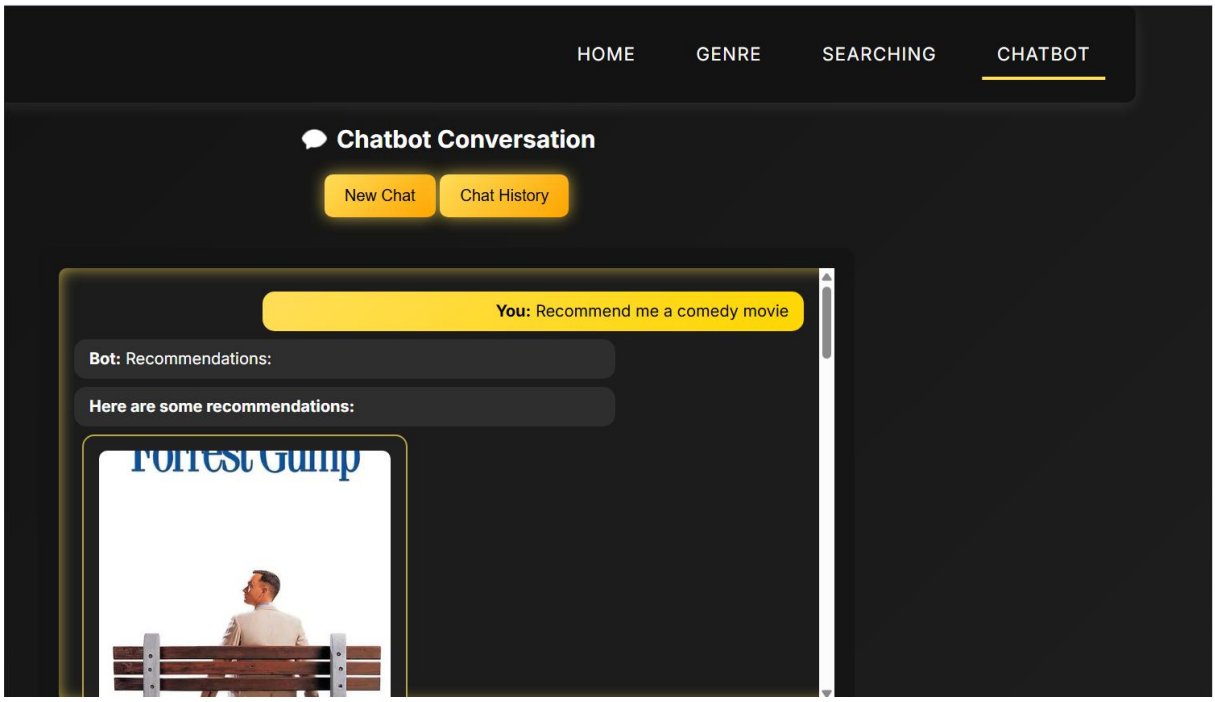
2. User interface and experience

a. Interface design:

- Chatbot:
  - Location: located on the navbar, main content area of the page
  - Design: Textbox input box with placeholder "Enter your movie request...", press “Enter” to submit.
  - Response: Displays bubble messages (user requests and system results).

- Display results:
  - Movie list: Carousel or grid format (5 movies/row) with poster, movie title, and rating (IMDb/Rotten Tomatoes).
  - Movie Details: Pop-up or subpage showing full information (description, actors, YouTube trailer).

- Search history: Displayed in a left panel, each item can be clicked to search again.



b. User experience

- Advantages:
  - Intuitive interaction: Chatbot simulates natural conversation, making it easy for users to enter requests.
  - Multimedia display: Movie posters and trailers increase appeal.
  - Clear error feedback: "No matching movie found" message with suggestions for movies with the highest imdb Rating.
  - Added ability to review search history and delete history if no longer needed.
- Limitations:
  - Response time: Average 10s for complex requests due to reliance on TMDB API and AI processing. Users may feel the wait is long.

VII. References

- https://platform.openai.com/docs/api-reference/usage
- https://cloud.google.com/discover/what-is-prompt-engineering
- https://www.datacamp.com/tutorial/fuzzy-string-python